

How I Built This Thing: KD CM Notes Tech Deep-Dive

Heads up: I'm not a real programmer. I'm a case manager who got tired of clicking the same buttons all day and asked AI to help me build a Chrome extension about it. This is me trying to explain what the heck I built and how it all kinda works. If you're also a non-coder reading this — hi, you're in the right place.

Table of Contents

1. [How This Whole Mess Started](#)
 2. [What This Thing Actually Does](#)
 3. [How the Code Organized Itself \(I Didn't Plan This\)](#)
 4. [The IIFE Thing — Why Every File Starts the Same Way](#)
 5. [The GM * Shim — Tricking Chrome Into Acting Like Tampermonkey](#)
 6. [How Data Moves Around](#)
 7. [Shadow DOM — Why Salesforce Is a Pain to Work With](#)
 8. [Floating Windows — How All Those Panels Work](#)
 9. [The Automation Stuff — How It Clicks Buttons for You](#)
 10. [What AI Was Actually Good At \(And What It Sucked At\)](#)
 11. ["Why Did My Extension Break?" — Lessons Learned the Hard Way](#)
 12. [Patterns I Noticed After the Fact](#)
 13. [Privacy & Security — Is This Thing Stealing Your Data?](#)
 14. [If I Had to Start Over \(What I'd Change\)](#)
 15. [Fancy Words Explained](#)
-

1. How This Whole Mess Started

The Problem (aka Why I Was Mad at Salesforce)

I work as a Case Manager at a disability firm. We use **Salesforce Lightning** to manage cases. It's one of those "powerful but painful" tools — you know the type. Every day I was doing the same boring stuff:

- Open client record, scroll forever to find basic info (SSN, DOB, phone)
- Type the same info into PDF forms over and over
- Copy-paste medical providers one by one
- Log "Failed to Reach" entries 2-3 times because there's multiple tracking systems
- Look up SSA's website to find which field office is closest to the client

At some point I realized: I'm spending more time *using Salesforce* than *helping people*. That's backwards.

The "Wait, Can I Just..." Moment

I'd used **Tampermonkey** before — it's a browser extension that lets you run little custom scripts on websites. I had a few small scripts that saved me maybe 20 minutes a day. But I kept bumping into walls. Tampermonkey doesn't work great with Chrome's new rules (Manifest V3), has storage limits, and can't do things like download files or use Chrome's APIs.

So one day I just thought: *Could I build my OWN Chrome extension?*

The honest answer was: I had NO idea how. But maybe AI could help. Especially with transforming a monolithic js script into a full fledged chrome extension.

How AI and I Worked Together

I didn't write this whole thing by hand. I pretty much told various AIs (Gemini, Deepseek, Copilot, Claude, etc.) what I wanted in plain English, and we went back and forth until it worked. My workflow was basically:

1. **Say what I want** — "Hey AI, make a floating window that shows the client's phone number and address"
2. **Look at the code it gives me** — sometimes it was perfect, sometimes it was gibberish
3. **Try it in Chrome** — reload the extension, see if it crashes
4. **Fix what broke** — paste errors into AI, let it help debug
5. **Repeat forever** — add features, refactor when things got messy
6. **Ask AI to explain stuff** — "what does this line do?" when I was confused

Over time I actually started *understanding* the code instead of just copy-pasting. This document is basically me trying to explain what I learned.

2. What This Thing Actually Does

So **KD CM Notes** is a Chrome Extension that adds a bunch of floating windows and automation to Salesforce. Think of it as a "toolbar for your toolbar" — extra stuff floating on top of Salesforce that makes your life easier.

Here's Everything It Does

KD CM Notes			
—		Client Note Workspace	(Alt+1) - The main hub
—	—	Info Panel	(Alt+Q) - SSN, DOB, phone, address
—	—	SSA Panel	(Alt+S) - Find FO offices + contact info
—	—	DDS Panel	(Alt+D) - Find DDS offices
—	—	Case Data	(Alt+A) - Case status at a glance
—	—	Raw Harvest Viewer	(Alt+Shift+I) - Shows ALL scraped data (debug tool)
—			
—		Medical Providers	(Alt+2) - Provider info in card view
—		Medication Manager	(Alt+3) - Drug search via NIH
—		Fax Forms	(Alt+4) - Generate PDFs, auto-upload to iFax
—		IR Tool	(Alt+5) - Parses investigation reports
—		1696 Fee Agreement	(Alt+6) - PDF stamping tool
—			
—		Automation Tool (floating button)	
—	—	FTR Logger	- Logs "Failed to Reach" calls
—	—	NCL Automation	- Generates Notice of Contact Letters
—	—	Email/ SMS Composer	- Template-based emails and texts
—	—	Macro Recorder	(Alt+M) - Records clicks and plays them back
—			
—		OBS Recorder (floating button)	- Records calls via OBS
—		iFax Observer (floating button)	- Watches for fax receipts in Outlook
—		Dashboard	(Alt+T) - Search notes, settings, backups
—		Scheduler	(Alt+L) - Calendar with reminders
—		Global Notes	(Alt+~) - Sticky notes that survive page reloads

Where It Runs

- **Salesforce pages** — `lightning.force.com` and `my.site.com`
- **iFax upload page** — `ifax.pro/sent/create/`
- **Outlook Web** — `outlook.cloud.microsoft/mail/` (for fax receipts)

3. How the Code Organized Itself (I Didn't Plan This)

The Happy Accident

I did NOT sit down and design a beautiful architecture before writing code. I just started building features one at a time, and eventually the project had like 40 files. At some point I looked at it and went "huh, this actually has layers."

What the AI Called the "4 Layers"

Looking back, the code kinda sorted itself into:

```

Layer 1: The Foundation
├─ manifest.json      - Chrome config (tells Chrome what the extension does)
├─ gm-compat.js       - Makes Chrome pretend to be Tampermonkey
├─ content.js         - The starting point
├─ src/config/
│   ├─ Themes.js      - Colors and themes
│   └─ Styles.css      - All the visual styles

Layer 2: The Toolbox
├─ Utils.js           - Random helpful functions
├─ Scraper.js         - Reads data from Salesforce pages
├─ WindowManager.js   - Makes floating windows draggable
├─ PdfManager.js      - Helps generate PDFs
├─ SSADDataManager.js - Manages SSA office database
└─ DistanceCalculator.js - Calculates distances for the map

Layer 3: The Features
├─ features/client-note/ - Client workspace panels
├─ features/automation/ - Automation stuff
└─ ui/panels/           - Standalone tool windows

Layer 4: The UI Shell
├─ Taskbar.js         - Bottom bar with tab buttons
├─ Dashboard.js       - Main dashboard
├─ GlobalNotes.js     - Scratchpad sidebar
├─ Scheduler.js       - Calendar
└─ BackupManager.js   - Backup/restore

```

The Namespace Trick

Every single file starts the same way:

```

(function () {
  const app = window.CM_App = window.CM_App || {};
  // ... the actual code ...
  app.Core.Utils = Utils; // or whatever this file is
})();

```

It basically creates one big global object called `CM_App` and every file adds its piece to it. The AI suggested this pattern. It's not fancy but it works for a Chrome extension where you can't use normal `import` statements.

4. The IIFE Thing — Why Every File Starts the Same Way

What Even Is an IIFE?

IIFE = "Immediately Invoked Function Expression." That's a fancy way of saying "a function that runs as soon as the file loads."

```

(function () {
  'use strict';
  // All the code lives here
})();

```

The `()` at the end means it runs immediately. The function wrapper means variables inside don't leak out into the global scope. The only thing that "escapes" is what we explicitly attach to `CM_App`.

The "Loading Order" Nightmare

Because we're not using a bundler (like Webpack or Vite), the order that scripts load in `manifest.json` is SUPER important. If `Utils.js` loads AFTER `AppObserver.js`, everything crashes because `AppObserver` tries to use `Utils` before it exists.

My manifest has like 30+ JS files in a specific order. Early on I'd add a new file, forget to put it in the right spot, and spend 20 minutes wondering why nothing worked.

The golden rule I learned: stuff that other files depend on has to go first.

1. gm-compat.js	(everyone needs GM_getValue etc.)
2. Core files	(Utils, Scraper, etc.)
3. Libraries	(leaflet, pdf-lib)
4. Features	(panels, automation)
5. UI shell	(Taskbar, Dashboard)
6. content.js	(last — starts everything)

5. The GM_* Shim — Tricking Chrome Into Acting Like Tampermonkey

Why It Exists

Tampermonkey gives you special functions like GM_getValue, GM_setValue, GM_addStyle. When I moved from Tampermonkey to a real Chrome extension, those functions disappeared. So I had to recreate them.

The Clever Part

The file gm-compat.js does two things at once:

1. **Loads ALL saved data into memory** when the extension starts (takes a moment)
2. **Reads from memory** after that (which is instant — no waiting)

```
// Startup: load everything
chrome.storage.local.get(null, (items) => {
  Object.assign(_cache, items || {});
  _cacheReady = true; // now reads are instant
});

// Read: just check memory (no callback needed!)
window.GM_getValue = function (key, defaultValue) {
  if (key in _cache) return _cache[key];
  return defaultValue;
};

// Write: save to memory AND to Chrome storage
window.GM_setValue = function (key, value) {
  _cache[key] = value;
  chrome.storage.local.set({ [key]: value }); // "fire and forget"
};
```

The "fire and forget" part means we don't wait for Chrome to confirm the save. It just... sends it. If it fails (like if the extension got shut down), we silently ignore the error.

The Storage Limit Problem

Chrome's storage has about a 10MB limit normally. But we store:

- Client records
- Medical provider tables
- Email/SMS templates
- Fax logs (including full PDFs encoded as text)
- OBS settings

That adds up FAST. We had to request unlimitedStorage permission to get around the limit. AI helped me figure this out after I hit the wall mid-development.

6. How Data Moves Around

The Basic Flow

```
Salesforce webpage (the DOM)
  ↓
Scraper.js reads the data (harvestFields / getAllPageData / getSSDFormData)
  ↓
Saved to storage (GM_setValue — permanently stored)
  ↓
UI panels read from storage and display it
  ↓
User edits something → saved back to storage
```

How I Named Things in Storage

Every stored value has a prefix so I know what it's for:

Pattern	Example	What It's For
cn_<clientId>	cn_0012w000123ABC	Per-client basic data
cn_form_data_<clientId>	cn_form_data_0012w...	Per-client form fields
cn_med_table_<clientId>	cn_med_table_0012w...	Per-client medical providers
sn_global_*	sn_global_cm1	App-wide settings
sn_*	sn_templates	Shared data
def_pos_*	def_pos_MED	Saved window positions

cn_ = client note, sn_ = super note (I just made these up).

How It Knows Which Client You're On

Every Salesforce record has an 18-character ID. The extension grabs it from the URL:

```
https://something.lightning.force.com/lightning/r/Case/0012w000123ABC/view
                                ^^^^^^^^^^^^^^^^^
                                That's the client ID
```

When you switch to a different client, the `AppObserver` spots the URL change, grabs the new ID, and all panels reload with that client's data. Old client's data stays in storage untouched.

7. Shadow DOM — Why Salesforce Is a Pain to Work With

The Problem

Salesforce uses something called **Web Components** with **Shadow DOM**. It's basically a secret hidden layer of HTML that normal queries can't see into. So a normal `document.querySelector()` returns nothing for elements inside those components.

Most of the data we need is buried 3-4 layers deep inside these shadow roots. It's like the data is behind glass and you can't reach it with normal tools.

The Fix: queryDeep

The `Utils.js` file has a function that manually digs into every element, checks if it has a shadow root, and searches inside it:

```
queryDeep(selector, root = document) {
  // First, try the normal way
  let el = root.querySelector(selector);
  if (el) return el;

  // If not found, visit EVERY element and check for shadow roots
  const walker = document.createTreeWalker(root, NodeFilter.SHOW_ELEMENT, null, false);
  let node = walker.nextNode();
  while (node) {
    if (node.shadowRoot) {
      el = this.queryDeep(selector, node.shadowRoot); // RECURSION!
      if (el) return el;
    }
    node = walker.nextNode();
  }
  return null;
}
```

How Scraper.js Actually Gets Data

The scraper works by looking for label-value pairs on the page. Like finding where it says "SSN" and grabbing the value next to it.

The tricky part: Salesforce uses different labels on different page layouts. So the scraper tries multiple patterns:

```
'Engagement Date': [
  'Engagement Date', // first try this exact label
  'Intake Date',     // if not found, try this
  'Date of Engagement', // okay fine, try this
],
```

The Fragile Truth

DOM scraping is **brittle**. If Salesforce changes their HTML (which they do every quarter), stuff breaks. I learned to:

1. **Try multiple ways** to find each field
2. **Gracefully show "(empty)"** instead of crashing when a field isn't found
3. **Built a debug tool** (Alt+Shift+I) that shows ALL scraped data so I can see what changed
4. **Calculate "days since"** for dates — if a date is way off, it's easier to spot

8. Floating Windows — How All Those Panels Work

What's Going On

Every panel in CM Notes is just a `<div>` that's been styled to look like a window and positioned on top of the Salesforce page. They're not real windows or iframes — just fancy divs floating around.

The Window Manager

There's a file called `WindowManager.js` that handles:

- **Dragging** — click and hold the header to move windows around
- **Resizing** — drag the edges to make windows bigger/smaller
- **Z-index** — clicking a window brings it to the front
- **Saving position** — hold the minimize button for 0.4 seconds and it remembers where you left it

The Hold-to-Save Feature

This is one of those things that just worked out really well. Instead of having a "Save Layout" button somewhere, you just **hold the minimize button for half a second**. The window flashes to confirm, and from then on it opens in that position.

Tap the button (don't hold) → window hides. Hold the button → saves position. Simple.

How Dragging Works (in Simple Terms)

1. You click on the window header
2. The code notes where your mouse is relative to the window
3. As you move your mouse, the window follows
4. When you let go, it stays where you left it
5. If you held the button long enough, it saves that position

9. The Automation Stuff — How It Clicks Buttons for You

The FTR Logger (Most Complex Feature)

The **Failed to Reach (FTR) Logger** is probably the fanciest thing in here. Before, logging a failed call took like 5 minutes of clicking around. Now it takes 10 seconds.

Before (manual):

```
1. Click "Last Activity" button
2. Type subject: "Call to Client/FTR"
3. Type comment: "FTR CL @ 555-123-4567 - No answer"
4. Click Save
5. If you also called the witness, repeat steps 1-4
6. Go create a New Task
7. Type subject: "Rose Letter 01 - NC to Client"
8. Set due date to today
9. Set type to "Send Letter"
10. Assign to Rose Robot
11. Save task
12. Open email composer
13. Paste template...
```

After (with CM Notes):

1. Pick FTR **result from** dropdown
 2. Click **"▶ Run FTR Logger"**
 3. Check **the** pre-filled fields **in** Salesforce
 4. Click **"☒ Confirm & Save"**
- Everything **else** happens **by** itself

Step-by-Step Automation

The automation code is literally just a list of steps with waits between them:

```
async runNCL(clientId) {  
  await this.ncl_step1(); // Click "New Task" button  
  await Utils.delay(1500); // Wait for it to load  
  await this.ncl_step2(); // Fill in the fields  
  await Utils.delay(1500); // Wait again  
  await this.ncl_step3(); // Assign and save  
}
```

The delays are necessary because Salesforce is a Single Page App — when you click something, the page doesn't actually reload, it just updates parts of itself. So we have to wait for animations and API calls to finish.

Why Automations Break (And the Macro Recorder Fix)

The problem with automation: if Salesforce changes a button's label or CSS class, the automation can't find it anymore. This happened ALL THE TIME.

The **Macro Recorder** was built to solve this. Instead of using fixed selectors, it:

1. **Records** your actual clicks during recording mode
2. **Captures multiple attributes** of each element (like its title, its label, its role, etc.)
3. **During playback**, it scores ALL elements on the page and picks the one that matches the **MOST** attributes

```
// Each attribute has a score weight  
// aria-label = 100 points (very reliable)  
// title = 90 points  
// name = 70 points  
// placeholder = 60 points  
// css path = 20 points (least reliable)
```

So even if Salesforce changes the button's CSS class, the recorder can still find it by its label text or title.

The Safety Two-Step

One smart pattern that emerged: automation runs in **TWO** phases.

```
Phase 1: Fill in the fields but DON'T save  
→ Open the form  
→ Type the subject and comment  
→ Wait for user to check it  
→ Show "Confirm & Save" button  
  
Phase 2: Actually save and do the rest  
→ Click Save  
→ If witness was called: do it again for witness  
→ If checked: send NCL, Email, SMS automatically
```

This way the user can review before anything actually gets saved to Salesforce.

10. What AI Was Actually Good At (And What It Sucked At)

What AI Was Great For

Thing	How It Helped
Boring repetitive code	The IIFE wrappers, namespace stuff, boring DOM queries
Explaining hard concepts	"Explain how the Haversine formula works and write it for me"
Debugging	"Why does this selector return nothing?" — AI spots the Shadow DOM issue
Rewriting code	"Take this 200-line mess and split it into separate files"
CSS and layout	Flexbox, animations, styling — I hate doing this manually

Documentation	Writing JSDoc comments and these very docs
Regex magic	Phone formatting, date parsing — I can never get this right
Migration help	Moving from Tampermonkey to Chrome Extension Manifest V3

What AI Was Bad At

Thing	Why It Sucked
Our specific Salesforce setup	AI doesn't know the exact HTML on our Salesforce instance
Cross-module bugs	Changing one file breaks another — AI doesn't see the big picture
Chrome extension weirdness	Extension context gets invalidated, service workers die randomly
Speed issues	AI suggested patterns that work but are slow (like querying the DOM 50 times)
Remembering all the files	After 50+ files, AI starts forgetting what connects to what
Design decisions	AI can give options but can't decide what's best for a case manager's workflow
Real testing	AI can't actually click around in Salesforce to see if something works visually

How We Ended Up Working Together

1. **Describe the feature** in plain English
2. **Read the AI's code** — I learned to spot when something looked wrong
3. **Test in Chrome** — load extension, open Salesforce, see if it works
4. **Fix bugs together** — paste error into AI, ask "what does this mean?"
5. **Go back and forth** — refine, retest, repeat
6. **Ask for explanations** — "why does this line exist?"

At some point I went from "I have no idea what this does" to "I get it but I'd rather have AI write it because it's faster."

11. "Why Did My Extension Break?" — Lessons Learned the Hard Way

Problem #1: The Script Doesn't Reload on Page Change

What happened: Salesforce is a "Single Page App" — when you navigate to a different client, the page doesn't actually reload. So my content script stayed loaded but the DOM was completely different.

Fix: Poll the URL every 500ms. If it changes, grab the new Client ID and re-initialize everything.

Lesson: SPAs are sneaky. Polling is ugly but it works.

Problem #2: Storage Ran Out

What happened: Fax logs with PDFs encoded as text take up MASSIVE space. Hit Chrome's storage limit.

Fix:

- Capped logs at 50 entries (oldest gets deleted first)
- Used `unlimitedStorage` permission
- Old PDFs just get lost after 50 entries

Lesson: Chrome storage is not a database. Don't treat it like one.

Problem #3: Salesforce Updates Broke Everything

What happened: Every 3 months Salesforce updates their UI and half my selectors stopped working.

Fix:

- Multiple fallback selectors for each field
- The Macro Recorder (scored matching, not exact paths)
- The debug tool to see what changed

Lesson: If you're scraping a website you don't own, expect it to break.

Problem #4: The Background Worker Dies Randomly

What happened: Chrome's Manifest V3 has a "service worker" instead of a persistent background page. Chrome can kill it whenever it wants. Timers stop working.

Fix:

- Used `chrome.alarms` (which survives worker restarts) for the 2-minute fax check
- Most logic lives in content scripts, not the background worker
- Background script is as simple as possible

Lesson: Don't trust the background worker. Put your logic in content scripts.

Problem #5: Listeners Stacked Up and Caused Chaos

What happened: Every time the Automation Panel re-rendered, it registered a new listener for data changes. Old listeners were never cleaned up. After switching tabs a few times, there were 10+ listeners all firing at once.

Fix: Save the listener ID and remove the old one before adding a new one.

```
// Before re-adding, remove the old one
if (this._valueListenerId !== null) {
  GM_removeValueChangeListener(this._valueListenerId);
}
// Now add the new one
this._valueListenerId = GM_addValueChangeListener(..., callback);
```

Lesson: Listeners are like guests at a party — if you don't kick the old ones out, they'll keep showing up.

Problem #6: The WebSocket Race Condition

What happened: The OBS companion app connects via WebSocket. If the connection drops and reconnects quickly, old callbacks could fire after new ones were set up, corrupting state.

Fix: Give each connection a unique ID. Callbacks check if they're still the "current" connection before doing anything.

```
const wsId = Math.random();
this._companionWsIdentity = wsId;

conn.onclose = () => {
  if (this._companionWsIdentity !== wsId) return; // This is an old callback, ignore it
  // Otherwise, handle the close properly
};
```

Lesson: Identity tokens prevent stale callbacks from messing things up.

12. Patterns I Noticed After the Fact

Pattern 1: Every Module Is a Tiny App

Each feature module has its own `init()`, `create()`, `render()`, `destroy()` functions. They manage their own DOM, communicate through storage, and only share the core library (Utils, WindowManager, Scraper).

I didn't plan this. It just happened because each feature was built separately.

Pattern 2: Storage Is How Modules Talk to Each Other

Modules can't directly call each other's functions, so they talk through GM storage:

- **Panel A saves** → `GM_setValue('some_key', data)`
- **Panel B notices** → `GM_addValueChangeListener('some_key', callback)`

It's like leaving notes on a bulletin board. Not elegant, but it works.

Pattern 3: The Floating Trigger Button

Several features (OBS Recorder, iFax Observer, Macro Recorder) use a little pill-shaped button on the edge of the screen:

```
Before hover:  [ 📺 ]
After hover:   [ 📺 OBS Recorder ]
After click:   [panel opens]
```

The button is draggable, remembers its position, and expands on hover. It's a nice pattern for features you don't need all the time but want quick access to.

Pattern 4: Always Plan for Corrupted Data

When reading from storage, ALWAYS check that the data looks right before using it:

```
const stored = GM_getValue('sn_templates', {});
const safeData = {
  email: (stored.email && typeof stored.email === 'object') ? stored.email : {},
  sms: (stored.sms && typeof stored.sms === 'object') ? stored.sms : {}
};
```

This prevents crashes when stored data somehow gets corrupted (which happens more than you'd think).

Pattern 5: The Write-Through Cache

The GM compat shim loads everything into memory on startup. Reads are instant (from memory). Writes go to both memory AND Chrome storage. First load is slow, but everything after that is fast.

13. Privacy & Security — Is This Thing Stealing Your Data?

What It Stores

- Client names, contact info, case details (stuff you can already see in Salesforce)
- Medical provider info
- Your preferences and templates
- Fax history (including PDFs)

All of this stays on YOUR computer in Chrome's local storage. Nothing gets sent to me or to some random server.

What It Talks To Online

Service	What It Sends	Why
raw.githubusercontent.com	Nothing (reads a public database)	Downloads SSA office info
api.github.com	Your token (if you set one up)	Saves database edits
nominatim.openstreetmap.org	Office addresses	Converts addresses to map coordinates
clinicaltables.nlm.nih.gov	First few letters of drug names	Medication autocomplete
ifax.pro	Generated PDFs	Sends faxes
OBS WebSocket (localhost)	Nothing	Controls OBS recording
Companion app (localhost:8027)	Phone numbers	Detects call events

Chrome Permissions — Why They're Needed

Permission	Why
storage	Save all your data
activeTab	Read/write Salesforce pages
clipboardWrite	Copy stuff for you
unlimitedStorage	Fax PDFs take a lot of space
downloads	Export PDFs
alarms	Check for fax receipts every 2 min

The CSP Thing

Salesforce has strict security rules about what scripts can load. To work around this, I bundle all libraries locally (leaflet, pdf-lib, etc.) instead of loading them from the internet. The fax receipt observer also has to clean up Outlook email HTML before processing it, because Outlook emails contain tags that violate Salesforce's security rules.

14. If I Had to Start Over (What I'd Change)

Things I'd Do Differently

1. **Use a bundler** — Webpack or Vite would handle the loading order automatically. No more "why is my script undefined?" drama.
2. **TypeScript** — Not for the type checking (okay, also for that), but because the documentation built into TypeScript would save so many "why is this undefined" errors.
3. **Central event system** — Instead of modules shouting at each other through GM storage, a proper event bus would be cleaner.
4. **Hide our UI properly** — Our floating windows can accidentally get affected by Salesforce's CSS. Using Shadow DOM for our own UI would prevent that.
5. **Automated tests** — Right now "testing" means "load it up and see if it crashes." Actual tests would catch problems faster.

6. **Use the background worker more** — Most logic is in content scripts. Some of it could live in the service worker.

Things I'd Keep Exactly the Same

1. **The IIFE module pattern** — Simple, no build step needed, every JS dev can read it.
2. **Floating windows** — Draggable, resizable, remembers position. Users love this.
3. **Hold-to-save** — No extra save button needed. Just hold and it remembers.
4. **Scored matching for macros** — This was actually a clever solution to a hard problem.
5. *GM_compat shim** — Abstracting storage behind a simple API made everything else much simpler.
6. **Step-by-step async automation** — It's ugly but it's reliable. Matches how a human would do it.

15. Fancy Words Explained

Word	What It Actually Means
IIFE	A function that runs as soon as the file loads. Keeps variables contained so they don't cause chaos.
Chrome Extension	A little program that adds features to Chrome. Like an app but lives in your browser.
Content Script	The part of the extension that runs on web pages and can change them.
Service Worker	A background script that handles events even when you close the page. But Chrome can kill it anytime.
Manifest V3	Chrome's latest rules for extensions. More restrictive than the old rules.
DOM	Document Object Model — the tree of elements that makes up a webpage.
Shadow DOM	A hidden section of the DOM that normal searches can't reach. Salesforce loves this.
CSS Selector	A pattern to find elements, like <code>button[title="Save"]</code> to find the Save button.
SPA	Single Page Application — a website that doesn't actually reload when you navigate.
MutationObserver	A watcher that notices when the page changes and tells you about it.
Promise	A placeholder for a value you'll get later. Like ordering food — you get a receipt now, food later.
async/await	Writing code that waits for things without being confusing. Reads like normal step-by-step instructions.
Base64	Turning a file (like a PDF) into text so it can be stored in places that only accept text.
Geocoding	Turning an address into GPS coordinates for a map.
Haversine Formula	Math to calculate distance between two points on a round Earth.
WebSocket	A two-way connection that stays open, like a phone call instead of mailing letters back and forth.
Exponential Backoff	If something fails, wait longer before retrying. Fail? Wait 1s. Fail again? Wait 2s. Then 4s, 8s, etc.
CSP	Content Security Policy — security rules that block certain types of content. Helpful for security, annoying for devs.
Shim/Polyfill	Code that makes new features work in old environments. Or in our case, makes Tampermonkey APIs work in Chrome.
Fire-and-forget	Start something and don't wait for the result. Just send it and move on.
Z-Index	Which element appears on top when things overlap. Higher number = on top.

Final Thoughts

This whole thing started as a tiny Tampermonkey script to pre-fill one date field. It somehow grew into a ~40-file Chrome extension that touches almost everything I do at work.

I never wanted to become a programmer. I just wanted to stop doing boring stuff. AI was the thing that made it possible.

Along the way, these are the things that stuck with me:

- **Code doesn't have to be pretty.** Ugly code that works is way better than beautiful code that doesn't exist.
- **Patterns appear on their own.** You don't need to design everything upfront. Just start building and patterns will emerge.
- **AI is a helper, not a magician.** The best results came when I understood enough to guide it, not when I blindly accepted whatever it gave me.
- **The hard part isn't coding.** It's knowing what you want and describing it clearly enough that an AI (or another person) can help you build it.

If you're reading this and you're not a developer and you're thinking "I could never build this" — you probably could. You just need:

1. A problem that annoys you enough
2. Patience to keep trying when things break
3. Willingness to read code even when it hurts your brain

4. An AI buddy to handle the parts you don't know yet

Last updated: June 2026

Written by: Kant Nguyen — Case Manager. Not a software engineer. Just a guy who was annoyed enough to learn.